

NEGATIVE-WEIGHT SINGLE-SOURCE
SHORTEST PATH USING BELLMAN-FORD &
MODIFIED DIJKSTRA'S

Jibran Sheikh - 08312

Daniyal Farooqui - 08320

MOTIVATION

- **Problem to Solve:** The challenge of finding the shortest paths in graphs with negative weights and detecting negative cycles.
- **Importance:** This problem is crucial for applications like network routing, financial systems, and optimization in large-scale systems.
- **Real-World Impact:** Affects industries that rely on graph-based models, such as logistics and telecommunications, where accurate pathfinding can reduce costs and increase efficiency.

PROBLEM STATEMENT

- **Definition:** Finding the shortest paths from a single source node to all other nodes in a graph, with potentially negative edge weights, while detecting and handling negative cycles.
- **Assumptions:** The graph is directed, and edge weights may be negative.
- **Constraints:** Efficient computation is required, particularly for large graphs.

RELATED WORK / RELATED ALGORITHMS

- Related Work: Bellman-Ford algorithm, Dijkstra's algorithm, and min-cost flow reductions are common methods used for solving shortest path problems with negative weights.
- Limitations:
- Bellman-Ford is computationally expensive, with a time complexity of $O(V \times E)$.
- Dijkstra's algorithm cannot handle negative edge weights.
- Our Approach: Combines Bellman-Ford with recursive scaling and decomposition to handle negative weights efficiently.

PROPOSED SOLUTION

- **Novelty:** We combine price functions (Johnson's algorithm), low-diameter decomposition with Bellman-Ford for improved performance.
- **Core Idea:** Initially, detect negative cycles using Bellman-Ford, then modify the graph to remove negative cycles, followed by shortest path calculations.
- **High-Level Diagram:** Show a flowchart of the algorithm's steps (Cycle Detection → Graph Modification → Shortest Path Calculation).

ALGORITHM OVERVIEW

Name:

Modified Bellman-Ford with Negative Edges and Dijkstra's comparison.

Pseudocode:

- Detect negative cycles using Bellman-Ford
- Reweight edges to eliminate negative weights
- Apply Dijkstra's algorithm on reweighted graph
- Convert distances back to original graph

ALGORITHM OVERVIEW

JOHNSONAPSP(V, E, w) :

⟨⟨Add an artificial source⟩⟩

add a new vertex s

for every vertex v

add a new edge $s \rightarrow v$

$w(s \rightarrow v) \leftarrow 0$

⟨⟨Compute vertex prices⟩⟩

$dist[s, \cdot] \leftarrow \text{BELLMANFORD}(V, E, w, s)$

if BELLMANFORD found a negative cycle

fail gracefully

⟨⟨Reweight the edges⟩⟩

for every edge $u \rightarrow v \in E$

$w'(u \rightarrow v) \leftarrow dist[s, u] + w(u \rightarrow v) - dist[s, v]$

⟨⟨Compute reweighted shortest path distances⟩⟩

for every vertex u

$dist'[u, \cdot] \leftarrow \text{DIJKSTRA}(V, E, w', u)$

⟨⟨Compute original shortest-path distances⟩⟩

for every vertex u

for every vertex v

$dist[u, v] \leftarrow dist'[u, v] - dist[s, u] + dist[s, v]$

COMPLEXITY ANALYSIS

Time Complexity:

- $O(V^2 \times E)$ (due to Bellman-Ford for negative cycle detection).

Comparison:

- Highlight improvements in handling negative weights, but note that time complexity doesn't meet the near-linear expectation.

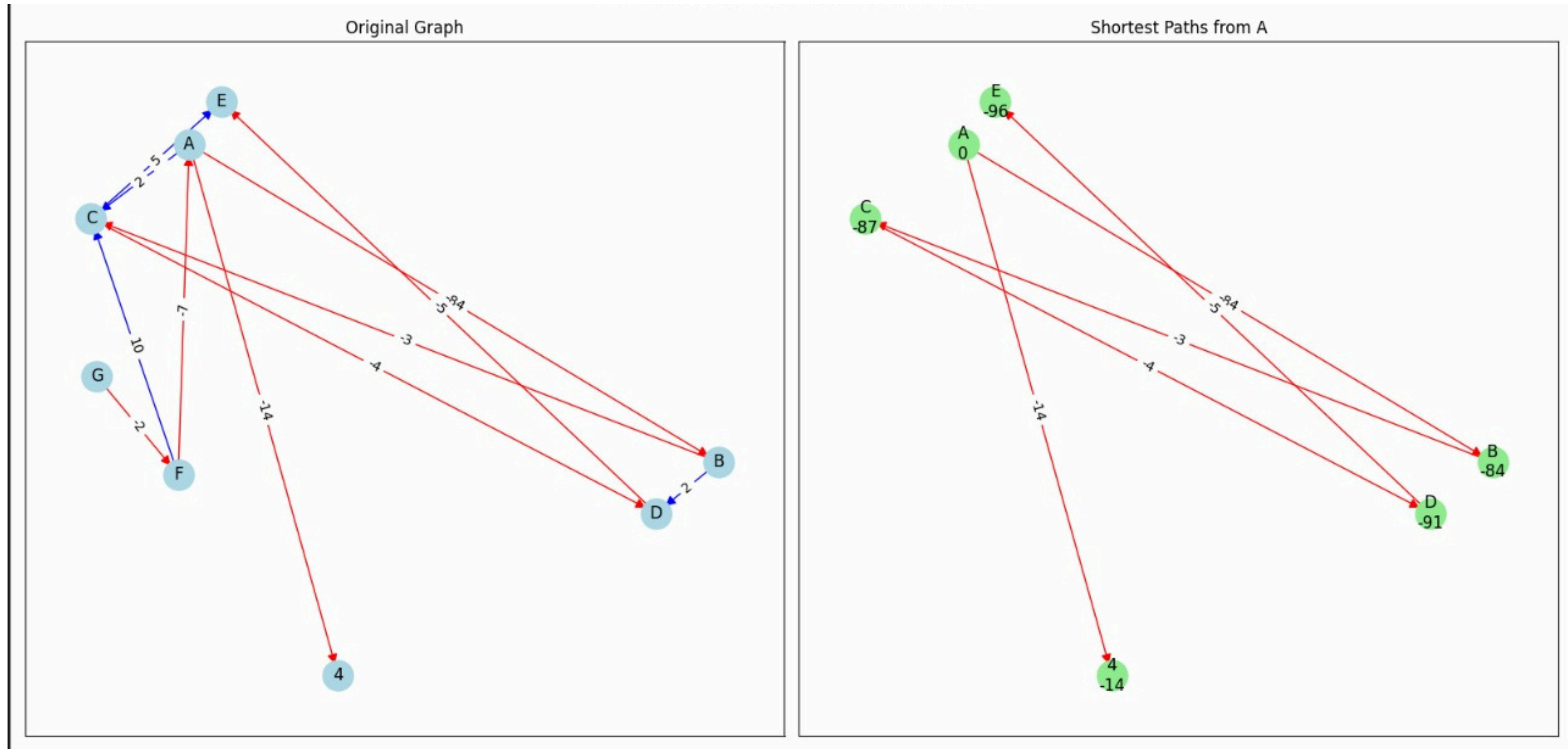
EXPERIMENTAL SETUP

- Datasets Used: Synthetic graphs with negative edge weights and negative cycles.
- Implementation: Python, using NetworkX for graph operations.
- Evaluation Metrics: Accuracy of shortest path calculation, runtime comparison with Bellman-Ford.

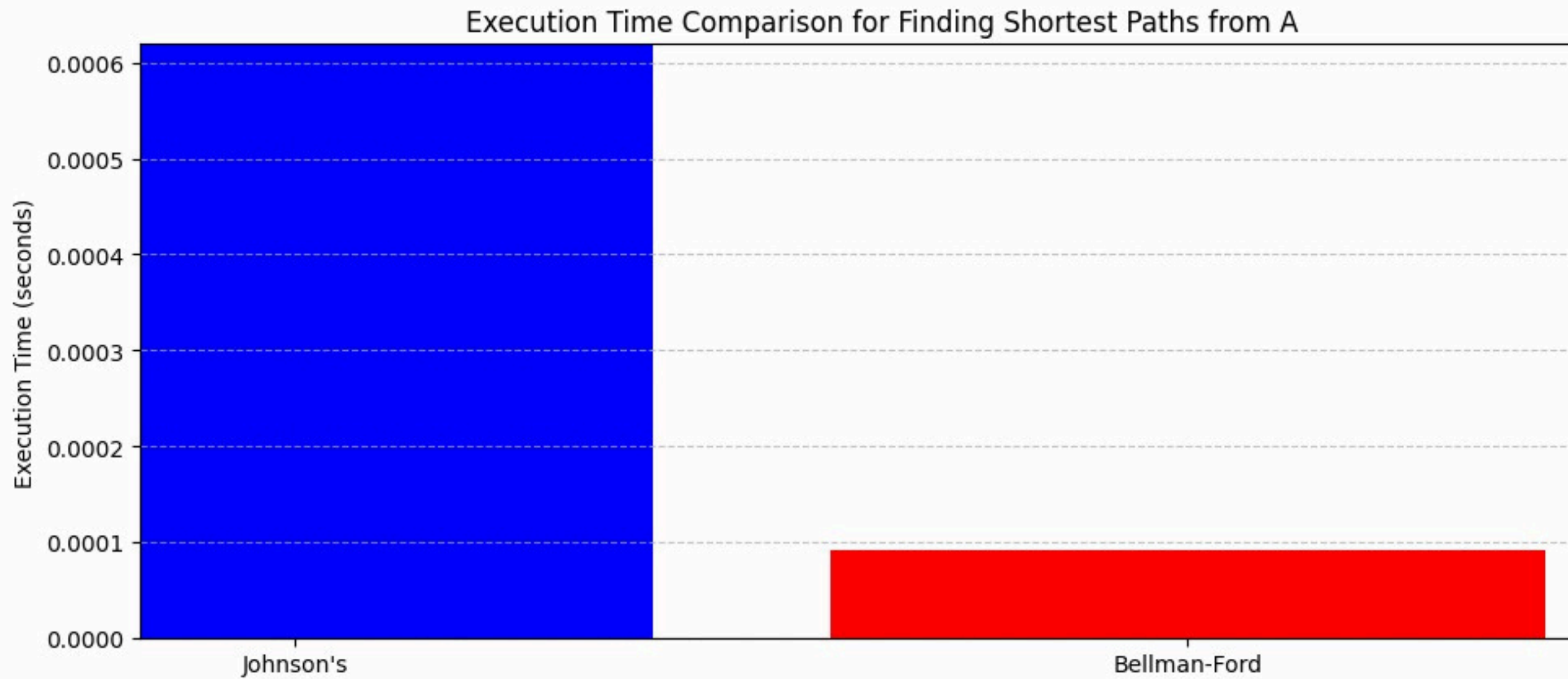
RESULTS

- Graph Comparison:
- Graph with Negative Cycle: Show original graph and modified graph after cycle removal.
- Graph without Negative Cycle: Show graph and shortest path results.
- Emphasize: The algorithm handles graphs with negative weights and detects cycles effectively.

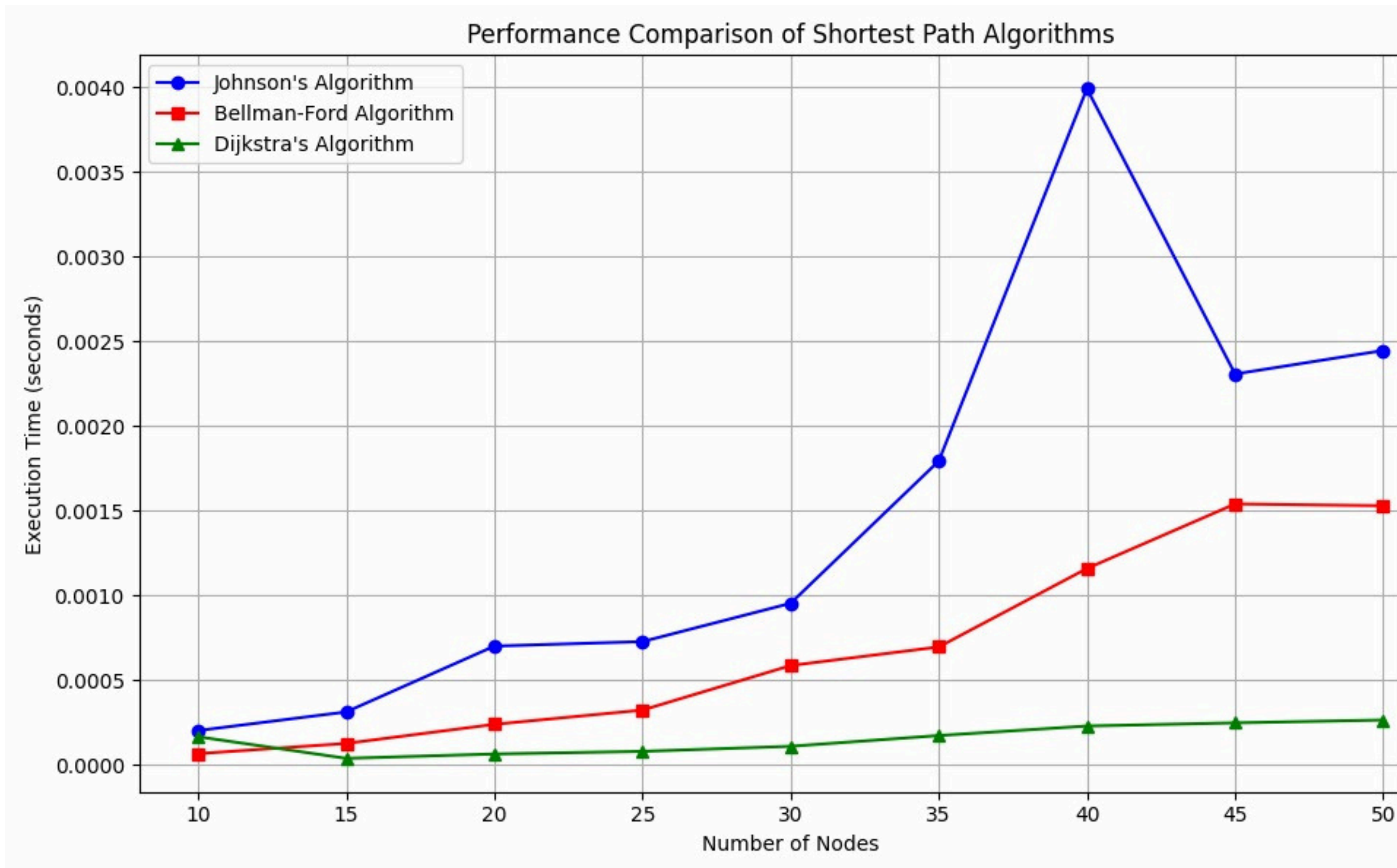
RESULTS



RESULTS



RESULTS



DEMO / EXAMPLE RUN

- Walkthrough: Demonstrate the algorithm on a small graph:
- Show how negative cycles are detected.
- Remove the negative cycle edges.
- Compute the shortest paths in the modified graph.
- Visuals: Use the visualizations from your code showing the original and modified graphs with highlighted negative edges.

LIMITATIONS / FUTURE DIRECTIONS

Limitations:

- High computational cost for large graphs.
- Recursive scaling and cycle detection could be optimized further.

Future Work:

- Investigate faster cycle detection algorithms.
- Explore parallelization for better performance.

CONCLUSION

Recap:

- Solved the negative-weight SSSP problem with improved cycle detection and graph modification.

Key Takeaway:

- The approach offers a more efficient way to handle negative cycles compared to traditional methods, but further optimizations are needed for near-linear time complexity.

REFERENCES

- Bellman-Ford Algorithm – Cormen, T.H., Leiserson, C.E., Rivest, R.L., Stein, C. "Introduction to Algorithms," 3rd Edition, MIT Press.
- NetworkX documentation for graph algorithms.



THANK YOU